

VLLM专题（二十四）—环境变量



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续费

👑 您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#) >

vLLM 使用以下环境变量来配置系统：

警告

请注意，VLLM_PORT 和 VLLM_HOST_IP 设置的是 vLLM 内部使用的端口和 IP，而不是 API 服务器的端口和 IP。如果您使用 `--host $VLLM_HOST_IP` 和 `--port $VLLM_PORT` 启动 API 服务器，它将无法正常工作。

vLLM 使用的所有环境变量都以 VLLM_ 为前缀。对于 Kubernetes 用户，请特别注意：不要将服务命名为 vllm，否则 Kubernetes 设置的环境变量可能与 vLLM 的环境变量发生冲突，因为 Kubernetes 会为每个服务设置以大写服务名为前缀的环境变量。

```
environment_variables: dict[str, Callable[[], Any]] = {

    # ===== Installation Time Env Vars =====

    # Target device of vLLM, supporting [cuda (by default),
    # rocm, neuron, cpu, openvino]
    "VLLM_TARGET_DEVICE":
        lambda: os.getenv("VLLM_TARGET_DEVICE", "cuda"),

    # Maximum number of compilation jobs to run in parallel.
    # By default this is the number of CPUs
    "MAX_JOBS":
        lambda: os.getenv("MAX_JOBS", None),

    # Number of threads to use for nvcc
    # By default this is 1.
    # If set, `MAX_JOBS` will be reduced to avoid oversubscribing the CPU.
    "NVCC_THREADS":
        lambda: os.getenv("NVCC_THREADS", None),

    # If set, vllm will use precompiled binaries (*.so)
    "VLLM_USE_PRECOMPILED":
        lambda: bool(os.getenv("VLLM_USE_PRECOMPILED")) or bool(
```

```

lambda: bool(os.environ.get("VLLM_USE_PRECOMPILED")) or bool(
    os.environ.get("VLLM_PRECOMPILED_WHEEL_LOCATION")),

# Whether to force using nightly wheel in python build.
# This is used for testing the nightly wheel in python build.
"VLLM_TEST_USE_PRECOMPILED_NIGHTLY_WHEEL":
lambda: bool(int(os.getenv("VLLM_TEST_USE_PRECOMPILED_NIGHTLY_WHEEL", "0")))
),

# CMake build type
# If not set, defaults to "Debug" or "RelWithDebInfo"
# Available options: "Debug", "Release", "RelWithDebInfo"
"CMAKE_BUILD_TYPE":
lambda: os.getenv("CMAKE_BUILD_TYPE"),

# If set, vllm will print verbose logs during installation
"VERBOSE":
lambda: bool(int(os.getenv('VERBOSE', '0'))),

# Root directory for vLLM configuration files
# Defaults to `~/.config/vllm` unless `XDG_CONFIG_HOME` is set
# Note that this not only affects how vllm finds its configuration files
# during runtime, but also affects how vllm installs its configuration
# files during **installation**.
"VLLM_CONFIG_ROOT":
lambda: os.path.expanduser(
    os.getenv(
        "VLLM_CONFIG_ROOT",
        os.path.join(get_default_config_root(), "vllm"),
    )),

# ===== Runtime Env Vars =====

# Root directory for vLLM cache files
# Defaults to `~/.cache/vllm` unless `XDG_CACHE_HOME` is set
"VLLM_CACHE_ROOT":
lambda: os.path.expanduser(
    os.getenv(
        "VLLM_CACHE_ROOT",
        os.path.join(get_default_cache_root(), "vllm"),
    )),

# used in distributed environment to determine the ip address
# of the current node, when the node has multiple network interfaces.
# If you are using multi-node inference, you should set this differently
# on each node.
'VLLM_HOST_IP':
lambda: os.getenv('VLLM_HOST_IP', ""),

```

```

# used in distributed environment to manually set the communication port
# Note: if VLLM_PORT is set, and some code asks for multiple ports, the
# VLLM_PORT will be used as the first port, and the rest will be generated
# by incrementing the VLLM_PORT value.
# '0' is used to make mypy happy
'VLLM_PORT':
lambda: int(os.getenv('VLLM_PORT', '0'))
if 'VLLM_PORT' in os.environ else None,

# path used for ipc when the frontend api server is running in
# multi-processing mode to communicate with the backend engine process.
'VLLM_RPC_BASE_PATH':
lambda: os.getenv('VLLM_RPC_BASE_PATH', tempfile.gettempdir()),

# If true, will load models from ModelScope instead of Hugging Face Hub.
# note that the value is true or false, not numbers
"VLLM_USE_MODELSCOPE":
lambda: os.environ.get("VLLM_USE_MODELSCOPE", "False").lower() == "true",

# Interval in seconds to log a warning message when the ring buffer is full
"VLLM_RINGBUFFER_WARNING_INTERVAL":
lambda: int(os.environ.get("VLLM_RINGBUFFER_WARNING_INTERVAL", "60")),

# path to cudatoolkit home directory, under which should be bin, include,
# and lib directories.
"CUDA_HOME":
lambda: os.environ.get("CUDA_HOME", None),

# Path to the NCCL library file. It is needed because nccl>=2.19 brought
# by PyTorch contains a bug: https://github.com/NVIDIA/nccl/issues/1234
"VLLM_NCCL_SO_PATH":
lambda: os.environ.get("VLLM_NCCL_SO_PATH", None),

# when `VLLM_NCCL_SO_PATH` is not set, vllm will try to find the nccl
# library file in the locations specified by `LD_LIBRARY_PATH`
"LD_LIBRARY_PATH":
lambda: os.environ.get("LD_LIBRARY_PATH", None),

# flag to control if vllm should use triton flash attention
"VLLM_USE_TRITON_FLASH_ATTN":
lambda: (os.environ.get("VLLM_USE_TRITON_FLASH_ATTN", "True").lower() in
         ("true", "1")),

# Force vllm to use a specific flash-attention version (2 or 3), only valid
# when using the flash-attention backend.
"VLLM_FLASH_ATTN_VERSION":
lambda: int(os.getenv('VLLM_FLASH_ATTN_VERSION', None))

```

```
lambda: maybe_convert_int(os.environ.get("VLLM_FLASH_ATTN_VERSION", None)),

# Internal flag to enable Dynamo fullgraph capture
"VLLM_TEST_DYNAMO_FULLGRAPH_CAPTURE":
lambda: bool(
    os.environ.get("VLLM_TEST_DYNAMO_FULLGRAPH_CAPTURE", "1") != "0"),

# Local rank of the process in the distributed setting, used to determine
# the GPU device id
"LOCAL_RANK":
lambda: int(os.environ.get("LOCAL_RANK", "0")),

# used to control the visible devices in the distributed setting
"CUDA_VISIBLE_DEVICES":
lambda: os.environ.get("CUDA_VISIBLE_DEVICES", None),

# timeout for each iteration in the engine
"VLLM_ENGINE_ITERATION_TIMEOUT_S":
lambda: int(os.environ.get("VLLM_ENGINE_ITERATION_TIMEOUT_S", "60")),

# API key for vLLM API server
"VLLM_API_KEY":
lambda: os.environ.get("VLLM_API_KEY", None),

# S3 access information, used for tensorizer to load model from S3
"S3_ACCESS_KEY_ID":
lambda: os.environ.get("S3_ACCESS_KEY_ID", None),
"S3_SECRET_ACCESS_KEY":
lambda: os.environ.get("S3_SECRET_ACCESS_KEY", None),
"S3_ENDPOINT_URL":
lambda: os.environ.get("S3_ENDPOINT_URL", None),

# Usage stats collection
"VLLM_USAGE_STATS_SERVER":
lambda: os.environ.get("VLLM_USAGE_STATS_SERVER", "https://stats.vllm.ai"),
"VLLM_NO_USAGE_STATS":
lambda: os.environ.get("VLLM_NO_USAGE_STATS", "0") == "1",
"VLLM_DO_NOT_TRACK":
lambda: (os.environ.get("VLLM_DO_NOT_TRACK", None) or os.environ.get(
    "DO_NOT_TRACK", None) or "0") == "1",
"VLLM_USAGE_SOURCE":
lambda: os.environ.get("VLLM_USAGE_SOURCE", "production"),

# Logging configuration
# If set to 0, vllm will not configure logging
# If set to 1, vllm will configure logging using the default configuration
# or the configuration file specified by VLLM_LOGGING_CONFIG_PATH
"VLLM_CONFIGURE_LOGGING":
```

```
lambda: int(os.getenv("VLLM_CONFIGURE_LOGGING", "1")),
"VLLM_LOGGING_CONFIG_PATH":
lambda: os.getenv("VLLM_LOGGING_CONFIG_PATH"),

# this is used for configuring the default logging level
"VLLM_LOGGING_LEVEL":
lambda: os.getenv("VLLM_LOGGING_LEVEL", "INFO"),

# if set, VLLM_LOGGING_PREFIX will be prepended to all log messages
"VLLM_LOGGING_PREFIX":
lambda: os.getenv("VLLM_LOGGING_PREFIX", ""),

# if set, vllm will call logits processors in a thread pool with this many
# threads. This is useful when using custom logits processors that either
# (a) launch additional CUDA kernels or (b) do significant CPU-bound work
# while not holding the python GIL, or both.
"VLLM_LOGITS_PROCESSOR_THREADS":
lambda: int(os.getenv("VLLM_LOGITS_PROCESSOR_THREADS", "0"))
if "VLLM_LOGITS_PROCESSOR_THREADS" in os.environ else None,

# Trace function calls
# If set to 1, vllm will trace function calls
# Useful for debugging
"VLLM_TRACE_FUNCTION":
lambda: int(os.getenv("VLLM_TRACE_FUNCTION", "0")),

# Backend for attention computation
# Available options:
# - "TORCH_SDPA": use torch.nn.MultiheadAttention
# - "FLASH_ATTN": use FlashAttention
# - "XFORMERS": use XFormers
# - "ROCM_FLASH": use ROCmFlashAttention
# - "FLASHINFER": use flashinfer
# - "FLASHMLA": use FlashMLA
"VLLM_ATTENTION_BACKEND":
lambda: os.getenv("VLLM_ATTENTION_BACKEND", None),

# If set, vllm will use flashinfer sampler
"VLLM_USE_FLASHINFER_SAMPLER":
lambda: bool(int(os.environ["VLLM_USE_FLASHINFER_SAMPLER"]))
if "VLLM_USE_FLASHINFER_SAMPLER" in os.environ else None,

# If set, vllm will force flashinfer to use tensor cores;
# otherwise will use heuristic based on model architecture.
"VLLM_FLASHINFER_FORCE_TENSOR_CORES":
lambda: bool(int(os.getenv("VLLM_FLASHINFER_FORCE_TENSOR_CORES", "0"))),
```

```
# Pipeline stage partition strategy
"VLLM_PP_LAYER_PARTITION":
lambda: os.getenv("VLLM_PP_LAYER_PARTITION", None),

# (CPU backend only) CPU key-value cache space.
# default is 4GB
"VLLM_CPU_KVCACHE_SPACE":
lambda: int(os.getenv("VLLM_CPU_KVCACHE_SPACE", "0")),

# (CPU backend only) CPU core ids bound by OpenMP threads, e.g., "0-31",
# "0,1,2", "0-31,33". CPU cores of different ranks are separated by '|'.
"VLLM_CPU_OMP_THREADS_BIND":
lambda: os.getenv("VLLM_CPU_OMP_THREADS_BIND", "all"),

# (CPU backend only) whether to use prepack for MoE layer. This will be
# passed to ipex.Llm.modules.GatedMLPMOE. On unsupported CPUs, you might
# need to set this to "0" (False).
"VLLM_CPU_MOE_PREPACK":
lambda: bool(int(os.getenv("VLLM_CPU_MOE_PREPACK", "1"))),

# OpenVINO device selection
# default is CPU
"VLLM_OPENVINO_DEVICE":
lambda: os.getenv("VLLM_OPENVINO_DEVICE", "CPU").upper(),

# OpenVINO key-value cache space
# default is 4GB
"VLLM_OPENVINO_KVCACHE_SPACE":
lambda: int(os.getenv("VLLM_OPENVINO_KVCACHE_SPACE", "0")),

# OpenVINO KV cache precision
# default is bf16 if natively supported by platform, otherwise f16
# To enable KV cache compression, please, explicitly specify u8
"VLLM_OPENVINO_CPU_KV_CACHE_PRECISION":
lambda: os.getenv("VLLM_OPENVINO_CPU_KV_CACHE_PRECISION", None),

# Enables weights compression during model export via HF Optimum
# default is False
"VLLM_OPENVINO_ENABLE_QUANTIZED_WEIGHTS":
lambda:
(os.environ.get("VLLM_OPENVINO_ENABLE_QUANTIZED_WEIGHTS", "0").lower() in
 ("on", "true", "1")),
# If the env var is set, then all workers will execute as separate
# processes from the engine, and we use the same mechanism to trigger
# execution on all workers.
# Run vLLM with VLLM_USE_RAY_SPMD_WORKER=1 to enable it.
"VLLM_USE_RAY_SPMD_WORKER":
lambda: bool(int(os.getenv("VLLM_USE_RAY_SPMD_WORKER", "0")))
```

```
lambda: bool(int(os.getenv("VLLM_USE_RAY_SPMD_WORKER", "0"))),

# If the env var is set, it uses the Ray's Compiled Graph
# (previously known as ADAG) API which optimizes the
# control plane overhead.
# Run vLLM with VLLM_USE_RAY_COMPILED_DAG=1 to enable it.
"VLLM_USE_RAY_COMPILED_DAG":
lambda: bool(int(os.getenv("VLLM_USE_RAY_COMPILED_DAG", "0"))),

# If the env var is set, it uses NCCL for communication in
# Ray's Compiled Graph. This flag is ignored if
# VLLM_USE_RAY_COMPILED_DAG is not set.
"VLLM_USE_RAY_COMPILED_DAG_NCCL_CHANNEL":
lambda: bool(int(os.getenv("VLLM_USE_RAY_COMPILED_DAG_NCCL_CHANNEL", "1"))
),

# If the env var is set, it enables GPU communication overlap
# (experimental feature) in Ray's Compiled Graph. This flag is ignored if
# VLLM_USE_RAY_COMPILED_DAG is not set.
"VLLM_USE_RAY_COMPILED_DAG_OVERLAP_COMM":
lambda: bool(int(os.getenv("VLLM_USE_RAY_COMPILED_DAG_OVERLAP_COMM", "0"))
),

# Use dedicated multiprocess context for workers.
# Both spawn and fork work
"VLLM_WORKER_MULTIPROC_METHOD":
lambda: os.getenv("VLLM_WORKER_MULTIPROC_METHOD", "fork"),

# Path to the cache for storing downloaded assets
"VLLM_ASSETS_CACHE":
lambda: os.path.expanduser(
    os.getenv(
        "VLLM_ASSETS_CACHE",
        os.path.join(get_default_cache_root(), "vllm", "assets"),
    ),
),

# Timeout for fetching images when serving multimodal models
# Default is 5 seconds
"VLLM_IMAGE_FETCH_TIMEOUT":
lambda: int(os.getenv("VLLM_IMAGE_FETCH_TIMEOUT", "5")),

# Timeout for fetching videos when serving multimodal models
# Default is 30 seconds
"VLLM_VIDEO_FETCH_TIMEOUT":
lambda: int(os.getenv("VLLM_VIDEO_FETCH_TIMEOUT", "30")),

# Timeout for fetching audio when serving multimodal models
# Default is 10 seconds
```

```
"VLLM_AUDIO_FETCH_TIMEOUT":
lambda: int(os.getenv("VLLM_AUDIO_FETCH_TIMEOUT", "10")),

# Cache size (in GiB) for multimodal input cache
# Default is 8GiB
"VLLM_MM_INPUT_CACHE_GIB":
lambda: int(os.getenv("VLLM_MM_INPUT_CACHE_GIB", "8")),

# Path to the XLA persistent cache directory.
# Only used for XLA devices such as TPUs.
"VLLM_XLA_CACHE_PATH":
lambda: os.path.expanduser(
    os.getenv(
        "VLLM_XLA_CACHE_PATH",
        os.path.join(get_default_cache_root(), "vllm", "xla_cache"),
    )),
"VLLM_FUSED_MOE_CHUNK_SIZE":
lambda: int(os.getenv("VLLM_FUSED_MOE_CHUNK_SIZE", "32768")),

# If set, vllm will skip the deprecation warnings.
"VLLM_NO_DEPRECATION_WARNING":
lambda: bool(int(os.getenv("VLLM_NO_DEPRECATION_WARNING", "0"))),

# If set, the OpenAI API server will stay alive even after the underlying
# AsyncLLMEngine errors and stops serving requests
"VLLM_KEEP_ALIVE_ON_ENGINE_DEATH":
lambda: bool(os.getenv("VLLM_KEEP_ALIVE_ON_ENGINE_DEATH", 0)),

# If the env var VLLM_ALLOW_LONG_MAX_MODEL_LEN is set, it allows
# the user to specify a max sequence length greater than
# the max length derived from the model's config.json.
# To enable this, set VLLM_ALLOW_LONG_MAX_MODEL_LEN=1.
"VLLM_ALLOW_LONG_MAX_MODEL_LEN":
lambda:
(os.environ.get("VLLM_ALLOW_LONG_MAX_MODEL_LEN", "0").strip().lower() in
 ("1", "true")),

# If set, forces FP8 Marlin to be used for FP8 quantization regardless
# of the hardware support for FP8 compute.
"VLLM_TEST_FORCE_FP8_MARLIN":
lambda:
(os.environ.get("VLLM_TEST_FORCE_FP8_MARLIN", "0").strip().lower() in
 ("1", "true")),
"VLLM_TEST_FORCE_LOAD_FORMAT":
lambda: os.getenv("VLLM_TEST_FORCE_LOAD_FORMAT", "dummy"),

# Time in ms for the zmq client to wait for a response from the backend
# server for incoming data generation
```



```
# server for simple data operations
"VLLM_RPC_TIMEOUT":
lambda: int(os.getenv("VLLM_RPC_TIMEOUT", "10000")),

# a list of plugin names to load, separated by commas.
# if this is not set, it means all plugins will be loaded
# if this is set to an empty string, no plugins will be loaded
"VLLM_PLUGINS":
lambda: None if "VLLM_PLUGINS" not in os.environ else os.environ[
    "VLLM_PLUGINS"].split(","),

# Enables torch profiler if set. Path to the directory where torch profiler
# traces are saved. Note that it must be an absolute path.
"VLLM_TORCH_PROFILER_DIR":
lambda: (None if os.getenv("VLLM_TORCH_PROFILER_DIR", None) is None else os
    .path.expanduser(os.getenv("VLLM_TORCH_PROFILER_DIR", "."))),

# If set, vLLM will use Triton implementations of AWQ.
"VLLM_USE_TRITON_AWQ":
lambda: bool(int(os.getenv("VLLM_USE_TRITON_AWQ", "0"))),

# If set, allow loading or unloading lora adapters in runtime,
"VLLM_ALLOW_RUNTIME_LORA_UPDATING":
lambda:
(os.environ.get("VLLM_ALLOW_RUNTIME_LORA_UPDATING", "0").strip().lower() in
    ("1", "true")),

# By default, vLLM will check the peer-to-peer capability itself,
# in case of broken drivers. See https://github.com/vllm-project/vllm/blob/a9b15c606f6ea67a072416ea0ea115261a2756058/vllm/distributed/device\_communicators/custom\_all\_reduce\_utils.py#L101-L108 for details. # noqa
# If this env var is set to 1, vLLM will skip the peer-to-peer check,
# and trust the driver's peer-to-peer capability report.
"VLLM_SKIP_P2P_CHECK":
lambda: os.getenv("VLLM_SKIP_P2P_CHECK", "0") == "1",

# List of quantization kernels that should be disabled, used for testing
# and performance comparisons. Currently only affects MPTLinearKernel
# selection
# (kernels: MacheteLinearKernel, MarlinLinearKernel, ExLlamaLinearKernel)
"VLLM_DISABLED_KERNELS":
lambda: [] if "VLLM_DISABLED_KERNELS" not in os.environ else os.environ[
    "VLLM_DISABLED_KERNELS"].split(","),

# If set, use the V1 code path.
"VLLM_USE_V1":
lambda: bool(int(os.getenv("VLLM_USE_V1", "1"))),
```

```
# Pad the fp8 weights to 256 bytes for ROCm
"VLLM_ROCM_FP8_PADDING":
lambda: bool(int(os.getenv("VLLM_ROCM_FP8_PADDING", "1"))),
# Divisor for dynamic key scale factor calculation for FP8 KV Cache
"K_SCALE_CONSTANT":
lambda: int(os.getenv("K_SCALE_CONSTANT", "200")),

# Divisor for dynamic value scale factor calculation for FP8 KV Cache
"V_SCALE_CONSTANT":
lambda: int(os.getenv("V_SCALE_CONSTANT", "100")),
# If set, enable multiprocessing in LLM for the V1 code path.
"VLLM_ENABLE_V1_MULTIPROCESSING":
lambda: bool(int(os.getenv("VLLM_ENABLE_V1_MULTIPROCESSING", "1"))),
"VLLM_LOG_BATCHSIZE_INTERVAL":
lambda: float(os.getenv("VLLM_LOG_BATCHSIZE_INTERVAL", "-1")),
"VLLM_DISABLE_COMPILE_CACHE":
lambda: bool(int(os.getenv("VLLM_DISABLE_COMPILE_CACHE", "0"))),

# If set, vllm will run in development mode, which will enable
# some additional endpoints for developing and debugging,
# e.g. `/reset_prefix_cache`
"VLLM_SERVER_DEV_MODE":
lambda: bool(int(os.getenv("VLLM_SERVER_DEV_MODE", "0"))),

# Controls the maximum number of requests to handle in a
# single asyncio task when processing per-token outputs in the
# V1 AsyncLLM interface. It is applicable when handling a high
# concurrency of streaming requests.
# Setting this too high can result in a higher variance of
# inter-message latencies. Setting it too low can negatively impact
# TTFT and overall throughput.
"VLLM_V1_OUTPUT_PROC_CHUNK_SIZE":
lambda: int(os.getenv("VLLM_V1_OUTPUT_PROC_CHUNK_SIZE", "128")),

# If set, vLLM will disable the MLA attention optimizations.
"VLLM_MLA_DISABLE":
lambda: bool(int(os.getenv("VLLM_MLA_DISABLE", "0"))),

# If set, vLLM will use the Triton implementation of moe_align_block_size,
# i.e. moe_align_block_size_triton in fused_moe.py.
"VLLM_ENABLE_MOE_ALIGN_BLOCK_SIZE_TRITON":
lambda: bool(int(os.getenv("VLLM_ENABLE_MOE_ALIGN_BLOCK_SIZE_TRITON", "0"))
),

# Number of GPUs per worker in Ray, if it is set to be a fraction,
# it allows ray to schedule multiple actors on a single GPU,
# so that users can colocate other actors on the same GPUs as vLLM.
```

```
"VLLM_RAY_PER_WORKER_GPUS":
lambda: float(os.getenv("VLLM_RAY_PER_WORKER_GPUS", "1.0")),

# Bundle indices for Ray, if it is set, it can control precisely
# which indices are used for the Ray bundle, for every worker.
# Format: comma-separated list of integers, e.g. "0,1,2,3"
"VLLM_RAY_BUNDLE_INDICES":
lambda: os.getenv("VLLM_RAY_BUNDLE_INDICES", ""),

# In some system, find_loaded_library() may not work. So we allow users to
# specify the path through environment variable VLLM_CUDART_SO_PATH.
"VLLM_CUDART_SO_PATH":
lambda: os.getenv("VLLM_CUDART_SO_PATH", None),

# Contiguous cache fetching to avoid using costly gather operation on
# Gaudi3. This is only applicable to HPU contiguous cache. If set to true,
# contiguous cache fetch will be used.
"VLLM_USE_HPU_CONTIGUOUS_CACHE_FETCH":
lambda: os.environ.get("VLLM_CONTIGUOUS_PA", "true").lower() in
("1", "true"),

# Rank of the process in the data parallel setting
"VLLM_DP_RANK":
lambda: int(os.getenv("VLLM_DP_RANK", "0")),

# World size of the data parallel setting
"VLLM_DP_SIZE":
lambda: int(os.getenv("VLLM_DP_SIZE", "1")),

# IP address of the master node in the data parallel setting
"VLLM_DP_MASTER_IP":
lambda: os.getenv("VLLM_DP_MASTER_IP", "127.0.0.1"),

# Port of the master node in the data parallel setting
"VLLM_DP_MASTER_PORT":
lambda: int(os.getenv("VLLM_DP_MASTER_PORT", "0")),

# Whether to use S3 path for model loading in CI via RunAI Streamer
"VLLM_CI_USE_S3":
lambda: os.environ.get("VLLM_CI_USE_S3", "0") == "1",

# Whether to use atomicAdd reduce in gptq/awq marlin kernel.
"VLLM_MARLIN_USE_ATOMIC_ADD":
lambda: os.environ.get("VLLM_MARLIN_USE_ATOMIC_ADD", "0") == "1",

# Whether to turn on the outlines cache for V0
# This cache is unbounded and on disk, so it's not safe to use in
# an environment with potentially malicious users.
```

```
# in an environment with potentially malicious users:  
"VLLM_V0_USE_OUTLINES_CACHE":  
lambda: os.environ.get("VLLM_V0_USE_OUTLINES_CACHE", "0") == "1",  
}
```